

UNIVERSITAS GUNADARMA



PRAKTIKUM KECERDASAN ARTIFICIAL

MANUAL BOOK

“Convolutional Neural Network Dalam Mendeteksi Kesehatan Lebah Madu”

Nama : Novacida Haqs
NPM : 51423115
Kelas : 3IA02
Fakultas : Teknologi Industri
Jurusan : Informatika
PJ : Yasin

Ditulis Guna Melengkapi Sebagian Syarat

Praktikum Kecerdasan Artificial

Universitas Gunadarma

2025

DAFTAR ISI

COVER	1
DAFTAR ISI	2
BAB I	3
1.1 Latar Belakang.....	3
1.2 Tujuan	4
BAB II	5
2.1 Convolutional Neural Network(CNN).....	5
2.2 Artificial Intelegence	6
2.3 Dataset.....	11
BAB III.....	14
ANALISA DAN PERANCANGAN.....	14
BAB IV	34
PENUTUP	34
4.1 Kesimpulan.....	35
4.2 Saran.....	35
DAFTAR PUSTAKA	36

BAB I

PENDAHULUAN

1.1 Latar Belakang

Lebah madu adalah salah satu penyerbuk terpenting untuk berbagai produk dalam rantai makanan. Saat ini, dengan bantuan teknologi yang berkembang, pengamatan terhadap pengendalian kesehatan lebah merupakan bidang studi yang sangat penting. Dalam penelitian ini, gambar yang diambil di lingkungan alami lebah diproses dengan arsitektur Convolutional Neural Network (CNN) dan status kesehatan lebah diklasifikasikan. Hasil yang diperoleh cukup menjanjikan untuk penelitian yang akan dilakukan di bidang ini. Selain itu, struktur arsitektur CNN dipelajari dan arsitektur CNN dengan jenis dan jumlah lapisan yang berbeda dibandingkan satu sama lain. Sebagai hasil perbandingan, menggunakan jumlah lapisan konvolusi yang ideal daripada menggunakan lapisan konvolusi dalam jumlah besar untuk arsitektur CNN, akan meningkatkan keberhasilan. Selain itu, penggunaan lapisan normalisasi yang berfungsi sebagai pendukung dalam arsitektur CNN terbukti sangat penting untuk meningkatkan keberhasilan.

Selama dekade terakhir, peningkatan dramatis dalam kemampuan komputasi pemrosesan chip dan biaya perangkat keras komputasi yang lebih rendah telah menyebabkan munculnya pembelajaran mendalam, yang mengacu pada sub-bidang pembelajaran mesin yang berfokus pada fitur pembelajaran yang diekstraksi dari data dan mengklasifikasikannya melalui beberapa lapisan dalam arsitektur hierarki jaringan saraf. Menggunakan jaringan saraf konvolusi (CNN) adalah salah satu metode pembelajaran mendalam yang paling menjanjikan untuk menangani beberapa tugas pengenalan pola. Namun, seperti kebanyakan jaringan saraf tiruan, CNN rentan terhadap multiple local optima. Oleh karena itu, untuk menghindari terjebak dalam local optima, diperlukan perbaikan pada CNN. Metode optimasi berdasarkan metaheuristik sangat ampuh dalam menyelesaikan masalah optimasi. Namun penelitian tentang penggunaan metaheuristik untuk mengoptimalkan CNN jarang dilakukan. Dalam karya ini, metode koloni lebah buatan (ABC), salah satu metode metaheuristik paling populer, diusulkan sebagai pendekatan alternatif untuk mengoptimalkan kinerja CNN. Dengan kata lain, kami bertujuan untuk meminimalkan kesalahan klasifikasi dengan menginisialisasi bobot pengklasifikasi CNN berdasarkan solusi yang dihasilkan oleh metode ABC. Selain itu, ABC terdistribusi juga disajikan sebagai metode untuk jumlah waktu yang diperlukan untuk menjalankan proses saat bekerja dengan kumpulan data pelatihan yang besar. Hasil percobaan menunjukkan bahwa metode yang diusulkan dapat meningkatkan kinerja CNN biasa baik dalam akurasi pengenalan dan waktu komputasi.

Convolutional Neural Network (CNN) telah menunjukkan kinerja yang sangat baik dalam banyak masalah visi komputer dan pembelajaran mesin. Banyak makalah yang solid telah diterbitkan mengenai topik ini, dan beberapa paket perangkat lunak CNN sumber terbuka berkualitas tinggi telah tersedia. Ada juga tutorial CNN atau manual perangkat lunak CNN yang ditulis dengan baik.

Diskriminasi subspecies lebah madu sangat penting untuk konservasi keanekaragaman hayati lebah madu. Identifikasi subspecies lebah madu (*A. mellifera*) Subspecies juga penting dalam pembiakan lebah madu karena banyak pembias yang tertarik memelihara stok murni dan sering kali memerlukan sertifikasi resmi untuk membuktikan bahwa lebah mereka termasuk dalam subspecies tertentu.

Pengetahuan tentang distribusi geografis subspecies lebah madu merupakan hasil dari berbagai penelitian yang terutama didasarkan pada morfometri. Dalam tulisan ini, saya mencoba menganalisis sifat-sifat jaringan saraf konvolusional. Model yang tertanam, transformasi hamburan diperkenalkan sebagai langkah pertama pemahaman operasi CNN. Kami melihat bahwa transformasi fitur dibangun di atas transformasi wavelet yang memisahkan variasi pada skala yang berbeda menggunakan transformasi wavelet. Analisis arsitektur CNN secara umum tidak dibahas dalam makalah ini, namun ini hanyalah analisis langkah pertama menuju matematika penuh pemahaman tentang jaringan saraf konvolusional.

1.2 Tujuan

Tujuan dari proyek ini adalah untuk mengembangkan model Convolutional Neural Network (CNN) yang dapat mengklasifikasikan gambar lebah berdasarkan dua kategori utama: subspecies dan kesehatan lebah. Proyek ini bertujuan untuk memanfaatkan teknik deep learning dalam mengidentifikasi dan memprediksi kondisi serta jenis subspecies lebah dari gambar, dengan tujuan mendukung penelitian dan pelestarian lebah yang penting untuk ekosistem dan pertanian.

BAB II

PEMBAHASAN

2.1 Convolutional Neural Network(CNN)

Berikut adalah langkah-langkah umum dalam mengembangkan model CNN untuk mendeteksi kesehatan lebah madu:

1. Preprocessing Data

Augmentasi Data: Menambahkan variasi pada dataset melalui rotasi, flipping, zooming, atau perubahan kecerahan untuk meningkatkan ketahanan model terhadap variasi gambar.

Normalisasi: Menyesuaikan nilai piksel gambar agar berada dalam rentang 0 hingga 1 untuk mempercepat konvergensi model.

2. Arsitektur CNN

Input Layer: Mengambil gambar lebah dengan ukuran tertentu

Convolutional Layer: Menggunakan beberapa filter untuk mengekstraksi fitur gambar, seperti tepi atau tekstur tubuh lebah yang menunjukkan gejala penyakit.

Pooling Layer: Mengurangi dimensi data agar proses komputasi lebih efisien dan menghindari overfitting.

Fully Connected Layer: Menggabungkan semua fitur yang diekstraksi dan mengklasifikasikannya menjadi beberapa kategori.

Output Layer: Menggunakan fungsi aktivasi softmax untuk menghasilkan probabilitas setiap kategori.

3. Training dan Validasi Model

Training: Model dilatih dengan dataset yang telah di-augmentasi. Optimizer seperti Adam digunakan untuk mempercepat proses pembelajaran.

Validasi: Performa model diuji pada dataset validasi untuk memantau akurasi dan mencegah overfitting.

4. Evaluasi Model

Menggunakan metrik seperti akurasi, precision, recall, dan F1-score untuk mengukur performa model pada dataset uji (test set).

Confusion Matrix dapat digunakan untuk melihat distribusi prediksi model pada setiap kelas.

Keuntungan dan Tantangan Menggunakan CNN

Keuntungan :

- Akurasi Tinggi: CNN sangat efektif dalam mengenali pola visual kompleks yang tidak mudah dikenali oleh manusia.
- Automasi: CNN mampu mengotomatisasi proses diagnosis yang biasanya membutuhkan keahlian khusus.
- Generalizability: Dengan augmentasi data, CNN bisa lebih tahan terhadap variasi gambar dari berbagai kondisi lapangan.

Tantangan :

- Data Berkualitas: Membutuhkan dataset gambar yang berkualitas tinggi dan terannotasi dengan baik.
- Overfitting: Jika dataset tidak cukup besar atau beragam, model dapat dengan mudah melakukan overfitting, di mana model belajar terlalu spesifik pada data latih dan gagal mengenali data baru.
- Komputasi: Pelatihan model CNN memerlukan daya komputasi yang besar, terutama saat menggunakan dataset dengan resolusi tinggi.

2.2 Artificial Intelligence

AI dibagi menjadi tiga kategori utama berdasarkan tingkat kecerdasannya:

Artificial Narrow Intelligence (ANI) atau AI Lemah: Merupakan AI yang dirancang untuk melakukan tugas spesifik, seperti pengenalan suara, deteksi wajah, atau rekomendasi produk. Contohnya adalah Siri atau Google Assistant.

Artificial General Intelligence (AGI) atau AI Kuat: Merupakan AI yang memiliki kemampuan kognitif yang setara dengan manusia. AGI dapat memahami, belajar, dan menerapkan pengetahuan secara umum pada berbagai tugas. Namun, AGI saat ini masih dalam tahap penelitian dan belum terwujud sepenuhnya.

Artificial Superintelligence (ASI): Merupakan bentuk AI yang lebih cerdas dari manusia dalam semua aspek, termasuk kreativitas, penalaran, dan pemecahan masalah. ASI masih merupakan konsep teoritis dan belum tercapai dalam pengembangan AI saat ini.

Sub-bidang dalam AI

AI mencakup berbagai sub-bidang yang memungkinkan mesin untuk "belajar" dan "berpikir" seperti manusia:

Machine Learning (ML): Adalah teknik AI di mana sistem komputer belajar dari data tanpa diprogram secara eksplisit. ML menggunakan algoritma untuk menemukan pola dalam data dan membuat prediksi atau keputusan berdasarkan pola tersebut. Contoh penerapannya adalah rekomendasi produk di ecommerce dan klasifikasi email spam.

Deep Learning: Merupakan cabang dari machine learning yang menggunakan jaringan saraf tiruan (Artificial Neural Networks) dengan banyak lapisan (deep). Deep learning sangat efektif untuk mengenali gambar, suara, dan teks. Contoh penerapannya adalah pengenalan gambar oleh Google Photos dan deteksi objek dalam video oleh self-driving car.

Natural Language Processing (NLP): Adalah cabang AI yang berfokus pada interaksi antara komputer dan bahasa manusia. NLP memungkinkan mesin untuk memahami, menganalisis, dan menghasilkan bahasa alami. Contohnya adalah chatbots, Google Translate, dan analisis sentimen di media sosial.

Computer Vision: Adalah bidang yang memungkinkan mesin untuk memahami dan menafsirkan data visual dari dunia nyata. Dengan menggunakan teknik seperti image processing dan deep learning, AI dapat mengenali objek, wajah, atau mendeteksi anomali pada gambar dan video. Penerapannya meliputi sistem keamanan berbasis pengenalan wajah dan deteksi penyakit dalam pencitraan medis.

Robotics: Menggabungkan AI dengan teknik robotika untuk menciptakan mesin yang dapat melakukan tugas fisik secara otomatis. Contoh penerapannya adalah robot industri yang digunakan untuk manufaktur dan robot penyelamat yang mampu beroperasi di lingkungan berbahaya.

Teknik dan Algoritma AI

Ada berbagai teknik dan algoritma yang digunakan dalam pengembangan AI, di antaranya:

Algoritma Klasifikasi: Digunakan untuk memprediksi kategori atau kelas suatu data. Contoh: Decision Trees, Random Forest, dan Support Vector Machines.

Regresi: Digunakan untuk memprediksi nilai berkelanjutan berdasarkan data input. Contoh: Linear Regression, Logistic Regression.

Clustering: Teknik untuk mengelompokkan data yang memiliki kesamaan. Contoh: K-Means Clustering, DBSCAN.

Jaringan Saraf Tiruan (Artificial Neural Networks): Merupakan model yang terinspirasi oleh cara kerja otak manusia. Jaringan ini terdiri dari lapisan neuron yang saling terhubung dan digunakan dalam deep learning.

Kategori AI

AI dibagi menjadi tiga kategori utama berdasarkan tingkat kecerdasannya:

Artificial Narrow Intelligence (ANI) atau AI Lemah: Merupakan AI yang dirancang untuk melakukan tugas spesifik, seperti pengenalan suara, deteksi wajah, atau rekomendasi produk. Contohnya adalah Siri atau Google Assistant.

Artificial General Intelligence (AGI) atau AI Kuat: Merupakan AI yang memiliki kemampuan kognitif yang setara dengan manusia. AGI dapat memahami, belajar, dan menerapkan pengetahuan secara umum pada berbagai tugas. Namun, AGI saat ini masih dalam tahap penelitian dan belum terwujud sepenuhnya. Artificial Superintelligence (ASI): Merupakan bentuk AI yang lebih cerdas dari manusia dalam semua aspek, termasuk kreativitas, penalaran, dan pemecahan masalah. ASI masih merupakan konsep teoritis dan belum tercapai dalam pengembangan AI saat ini.

Kelebihan AI

Efisiensi dan Produktivitas Tinggi

AI mampu memproses data dalam jumlah besar dengan cepat dan akurat. Hal ini meningkatkan efisiensi dalam berbagai tugas, seperti analisis data, pemrosesan gambar, dan pencarian informasi.

Contoh: Otomatisasi proses manufaktur menggunakan robot yang dipandu oleh AI dapat mengurangi kesalahan manusia dan meningkatkan produksi.

Otomatisasi Tugas Berulang

AI dapat mengotomatisasi tugas yang bersifat rutin dan berulang sehingga mengurangi beban kerja manusia dan memungkinkan mereka fokus pada tugas yang lebih kompleks dan kreatif.

Contoh: Chatbot AI yang digunakan dalam layanan pelanggan dapat menjawab pertanyaan umum secara otomatis, menghemat waktu dan tenaga manusia.

Pengambilan Keputusan yang Lebih Akurat

Dengan kemampuan analisis data yang mendalam, AI dapat memberikan wawasan yang lebih akurat untuk pengambilan keputusan. Sistem AI mampu mengidentifikasi pola dan tren dalam data yang sulit dilihat oleh manusia.

Contoh: Algoritma AI dalam analisis pasar saham dapat memprediksi pergerakan harga saham berdasarkan data historis dan tren pasar saat ini.

Personalisasi Layanan

AI memungkinkan personalisasi yang lebih baik dalam layanan, seperti rekomendasi produk, konten, atau iklan berdasarkan preferensi dan perilaku pengguna.

Contoh: Rekomendasi film di platform seperti Netflix atau produk di e-commerce seperti Amazon yang disesuaikan dengan preferensi pengguna.

Kemampuan Pembelajaran dan Adaptasi

Dengan teknik Machine Learning, sistem AI dapat belajar dari data yang ada dan memperbaiki performanya seiring waktu tanpa intervensi manusia secara langsung.

Contoh: Sistem deteksi penipuan yang menggunakan AI dapat belajar mengenali pola baru yang muncul dalam upaya penipuan berdasarkan data transaksi sebelumnya.

Pengurangan Human Error

AI bekerja berdasarkan data dan algoritma, sehingga mengurangi risiko kesalahan yang mungkin terjadi karena faktor manusia seperti kelelahan atau kesalahan input.

Contoh: Penggunaan AI dalam diagnosis medis dapat mengurangi kesalahan dalam interpretasi hasil pencitraan medis seperti MRI atau CT scan.

Kekurangan AI

Ketergantungan pada Data AI sangat bergantung pada data yang berkualitas untuk memberikan hasil yang akurat. Data yang bias, tidak lengkap, atau tidak representatif dapat menghasilkan keputusan yang salah atau tidak adil.

Contoh: Jika data pelatihan untuk sistem rekrutmen AI mengandung bias gender, maka sistem dapat mereplikasi bias tersebut dalam proses seleksi kandidat.

Kurangnya Pemahaman Konteks

Meskipun AI dapat menganalisis data dan mengidentifikasi pola, AI sering kali tidak memiliki pemahaman konteks atau makna di balik data. Hal ini membuat AI kurang efektif dalam tugas yang memerlukan intuisi atau pemahaman manusia.

Contoh: Sistem AI mungkin kesulitan memahami sarkasme atau humor dalam teks yang dianalisis oleh model Natural Language Processing (NLP).

Biaya Pengembangan yang Tinggi, Pengembangan sistem AI yang efektif membutuhkan biaya yang besar untuk infrastruktur komputasi, data, dan keahlian teknis. Implementasi AI juga memerlukan pemeliharaan dan pembaruan yang berkelanjutan.

Contoh: Perusahaan yang ingin menggunakan AI dalam proses bisnisnya harus berinvestasi dalam server GPU yang mahal dan mempekerjakan data scientist yang ahli.

Masalah Etika dan Privasi, AI dapat menimbulkan masalah etika terkait dengan pengumpulan dan penggunaan data pribadi. Sistem pengenalan wajah atau analisis data pengguna dapat mengancam privasi individu jika tidak ada regulasi yang memadai.

Contoh: Penggunaan AI untuk pengawasan oleh pemerintah dapat mengancam kebebasan privasi warga negara jika digunakan tanpa pengawasan atau regulasi yang jelas. Penggantian Tenaga Kerja

Otomatisasi yang dihasilkan oleh AI dapat mengurangi kebutuhan akan pekerjaan yang bersifat rutin atau manual, mengakibatkan potensi pengangguran di sektor-sektor tertentu.

Contoh: Pekerjaan seperti kasir, operator produksi, dan telemarketing dapat digantikan oleh teknologi AI, mengurangi kebutuhan akan tenaga manusia di bidang tersebut.

Sulit Diprediksi dan Diterangkan (Black Box Problem)

Banyak model AI, terutama yang menggunakan deep learning, sulit dijelaskan cara kerjanya. Ini disebut sebagai "black box problem", di mana keputusan yang dibuat AI sulit dipahami oleh manusia.

Contoh: Dalam aplikasi medis, dokter mungkin tidak dapat menjelaskan secara jelas bagaimana model deep learning mencapai kesimpulan tertentu dari data pasien.

Kesimpulan

AI menawarkan berbagai kelebihan seperti peningkatan efisiensi, otomatisasi, dan kemampuan analitik yang kuat. Namun, AI juga memiliki kekurangan seperti ketergantungan pada data, biaya pengembangan yang tinggi, dan tantangan etika. Dengan mempertimbangkan kelebihan dan kekurangan tersebut, penting bagi pengembang dan pengguna untuk mengimplementasikan AI dengan hati-hati, memastikan penggunaan yang bertanggung jawab, dan mengikuti regulasi yang ada agar manfaatnya dapat dinikmati secara maksimal tanpa menimbulkan dampak negatif yang signifikan.

2.3 Dataset

Dataset dapat dikategorikan berdasarkan berbagai aspek, seperti tipe data, sumber data, atau tujuan penggunaannya. Berikut beberapa jenis dataset yang umum:

Berdasarkan Sumber Data

Dataset Internal: Data yang dikumpulkan secara langsung oleh organisasi atau individu, seperti data penjualan, data pelanggan, dan data keuangan.

Dataset Eksternal: Data yang diperoleh dari sumber luar, seperti data dari lembaga pemerintah (misalnya BPS), dataset publik (misalnya dari Kaggle), atau data yang diperoleh melalui API dari platform seperti Twitter atau Google Maps.

Berdasarkan Tipe Data

Dataset Numerik: Berisi data yang bersifat numerik atau angka, seperti pendapatan, suhu, dan jumlah penjualan.

Dataset Kategorikal: Berisi data yang berbentuk kategori atau label, seperti jenis kelamin (pria/wanita), warna (merah, hijau, biru), dan status kesehatan (sehat/sakit).

Dataset Teks: Berisi data berupa teks, seperti artikel, review, atau tweet.

Dataset Gambar: Berisi data berupa gambar atau foto, sering digunakan dalam tugas computer vision, seperti pengenalan wajah atau deteksi objek.

Dataset Audio: Berisi data berupa rekaman suara atau audio, digunakan dalam tugas seperti pengenalan suara atau analisis sentimen audio.

Berdasarkan Tujuan Penggunaan

Dataset Pelatihan (Training Dataset): Digunakan untuk melatih model machine learning. Dataset ini merupakan bagian terbesar dari seluruh dataset.

Dataset Validasi (Validation Dataset): Digunakan untuk menguji model selama proses pelatihan untuk menghindari overfitting dan membantu dalam penyetelan hiperparameter.

Dataset Pengujian (Testing Dataset): Digunakan untuk menguji performa akhir model setelah selesai dilatih. Dataset ini tidak pernah digunakan selama proses pelatihan untuk memberikan evaluasi yang objektif.

Ciri-Ciri Dataset yang Baik

Kualitas Data yang Tinggi Data yang bersih dan bebas dari kesalahan, seperti duplikasi, nilai kosong (missing values), atau data yang salah (outliers). Contoh: Dalam dataset penjualan, semua nilai harga dan kuantitas harus valid (positif dan numerik).

Representatif Dataset harus mewakili populasi yang akan diprediksi oleh model. Data yang tidak representatif dapat menyebabkan bias dalam model. Contoh: Jika kita membuat model untuk memprediksi penyakit jantung pada populasi umum, maka dataset yang digunakan harus mencakup berbagai kelompok usia, jenis kelamin, dan kondisi kesehatan yang beragam.

Ukuran Dataset yang Memadai Ukuran dataset harus cukup besar untuk memungkinkan model machine learning belajar pola yang ada di data. Namun, ukuran yang terlalu besar dapat menyebabkan waktu pelatihan yang lebih lama dan membutuhkan lebih banyak sumber daya komputasi.

Contoh: Untuk tugas klasifikasi gambar sederhana, ribuan sampel mungkin cukup. Namun, untuk model deep learning yang kompleks, jutaan sampel mungkin diperlukan.

Variabilitas yang Cukup Dataset yang baik harus memiliki variasi yang cukup dalam fitur-fitur yang ada untuk memungkinkan model belajar pola yang kompleks.

Contoh: Dalam dataset gambar anjing dan kucing, harus ada variasi dalam ukuran, warna, posisi, dan latar belakang gambar untuk membantu model mengenali berbagai ciri hewan.

Tahapan Pengolahan Dataset

Pengumpulan Data Data dikumpulkan dari berbagai sumber, baik internal maupun eksternal, tergantung pada tujuan analisis. Contoh: Mengumpulkan data transaksi penjualan dari sistem ERP perusahaan atau mengunduh dataset gambar dari situs Kaggle.

Pembersihan Data (Data Cleaning)

Proses ini melibatkan penghapusan duplikasi, menangani missing values, mengoreksi kesalahan data, dan menangani outliers. Contoh: Mengganti nilai kosong dengan rata-rata atau median untuk kolom numerik atau menghapus sampel yang memiliki banyak nilai kosong.

Eksplorasi Data (Exploratory Data Analysis)

Analisis awal terhadap dataset untuk memahami distribusi data, korelasi antar variabel, dan pola-pola yang ada. Contoh: Menggunakan histogram, scatter plot, dan heatmap untuk memvisualisasikan data.

Transformasi Data (Data Transformation)

Mengubah data ke dalam format yang sesuai untuk analisis lebih lanjut atau pelatihan model.

Transformasi ini dapat meliputi normalisasi, standarisasi, encoding kategori, dan pembuatan fitur baru.

Contoh: Melakukan normalisasi pada kolom harga untuk mengubahnya ke rentang 0 hingga 1.

Pembagian Dataset

Dataset dibagi menjadi tiga bagian utama: training set, validation set, dan test set.

Contoh: 70% untuk training, 15% untuk validation, dan 15% untuk test.

Contoh Dataset pada Proyek Deteksi Kesehatan Lebah Madu

Pada proyek deteksi kesehatan lebah madu menggunakan CNN, dataset yang digunakan mungkin berisi gambar lebah dengan label yang menunjukkan kondisi kesehatannya, seperti:

Sehat, Terinfeksi Varroa, Terinfeksi Nosema Memiliki Sayap Deformasi

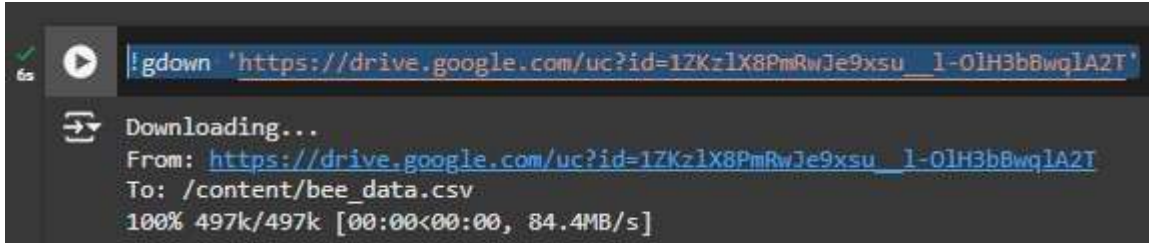
Dataset ini kemudian akan diolah dan digunakan untuk melatih model CNN untuk mengenali dan mengklasifikasikan kondisi kesehatan lebah berdasarkan gambar.

Kesimpulan

Dataset adalah komponen utama dalam proyek data science dan machine learning. Kualitas dan keragaman dataset sangat berpengaruh pada hasil analisis dan performa model yang dibangun. Oleh karena itu, penting untuk memilih, membersihkan, dan memproses dataset dengan cermat agar model yang dihasilkan dapat memberikan prediksi yang akurat dan andal.

BAB III

ANALISA DAN PERANCANG



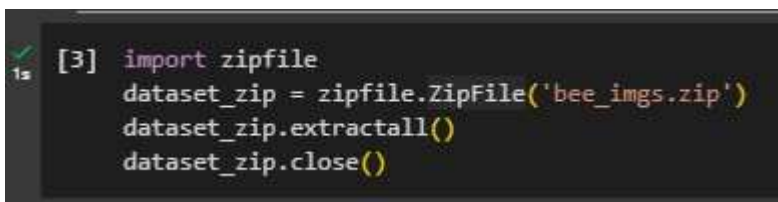
```
!gdown 'https://drive.google.com/uc?id=1ZKz1X8PmRwJe9xsu__1-0lH3b8wqlA2T'  
Downloading...  
From: https://drive.google.com/uc?id=1ZKz1X8PmRwJe9xsu__1-0lH3b8wqlA2T  
To: /content/bee_data.csv  
100% 497k/497k [00:00<00:00, 84.4MB/s]
```

- perintah ini akan mengunduh file yang memiliki ID tersebut dari Google Drive dan menyimpannya ke dalam lingkungan kerja (seperti Colab atau Jupyter).



```
[2] !gdown 'https://drive.google.com/uc?id=1w3w2ew0LYnxo76GPQmLF5011RwFk2w0T'  
Downloading...  
From (original): https://drive.google.com/uc?id=1w3w2ew0LYnxo76GPQmLF5011RwFk2w0T  
From (redirected): https://drive.google.com/uc?id=1w3w2ew0LYnxo76GPQmLF5011RwFk2w0T&confirm=t&uuid=6be7665f-3c65-452d-a003-a00a20c66966  
To: /content/bee_imgs.zip  
100% 52.9M/52.9M [00:01<00:00, 40.0MB/s]
```

- perintah ini akan mengunduh file yang memiliki ID tersebut dari Google Drive dan menyimpannya ke dalam lingkungan kerja (seperti Colab atau Jupyter).



```
[3] import zipfile  
dataset_zip = zipfile.ZipFile('bee_imgs.zip')  
dataset_zip.extractall()  
dataset_zip.close()
```

- Kode ini digunakan untuk membuka dan mengekstrak semua file dari arsip ZIP 'bee_imgs.zip', yang kemungkinan berisi dataset gambar lebah (berdasarkan nama file ZIP).

```

# Common Libraries
import pandas as pd
import numpy as np
import sys
import os
import random
from pathlib import Path

# Image processing
import imageio
import skimage
import skimage.io
import skimage.transform

# Charts
import matplotlib.pyplot as plt
import seaborn as sns

import scipy
from sklearn.model_selection import train_test_split
from sklearn import metrics

from keras import optimizers
from keras.models import Sequential
from keras.layers import Dense, Conv2D, Flatten, MaxPool2D, Dropout, BatchNormalization, LeakyReLU
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from keras.callbacks import ModelCheckpoint, Callback, EarlyStopping, ReduceLROnPlateau
from keras.utils import to_categorical
import matplotlib.pyplot as plt
import imageio.v2 as imageio

import tensorflow as tf

# Set random seed to make results reproducible
np.random.seed(42)
tf.random.set_seed(42)

# Global variables
img_folder='bee_imgs'
img_width=100
img_height=100
img_channels=3

```

- Kode ini merupakan setup awal untuk proyek pengenalan gambar menggunakan Convolutional Neural Network (CNN) dengan dataset gambar lebah. Kode ini mempersiapkan lingkungan dengan mengimpor library yang diperlukan dan mengatur variabel penting untuk proses preprocessing dan pelatihan model.

```
bees=pd.read_csv('bee_data.csv',
                 index_col=False,
                 dtype={'subspecies':'category', 'health':'category','caste':'category'})

# Will use this function later to load images of preprocessed bees
# Don't load images just from the start to save memory for preprocessing steps
def read_img(file):
    img = skimage.io.imread(img_folder + file)
    img = skimage.transform.resize(img, (img_width, img_height), mode='reflect')
    return img[:, :, :img_channels]

# Cannot impute nans, drop them
bees.dropna(inplace=True)

# Some image files don't exist. Leave only bees with available images.
img_exists = bees['file'].apply(lambda f: os.path.exists(img_folder + f))
bees = bees[img_exists]

bees.head()
```

	file	date	time	location	zip code	subspecies	health	pollen_carrying	caste
0	041_066.png	8/28/18	16:07	Alvin, TX, USA	77511	-1	hive being robbed	False	worker
1	041_072.png	8/28/18	16:07	Alvin, TX, USA	77511	-1	hive being robbed	False	worker
2	041_073.png	8/28/18	16:07	Alvin, TX, USA	77511	-1	hive being robbed	False	worker
3	041_067.png	8/28/18	16:07	Alvin, TX, USA	77511	-1	hive being robbed	False	worker
4	041_059.png	8/28/18	16:07	Alvin, TX, USA	77511	-1	hive being robbed	False	worker

Next steps: [Generate code with bees](#) [View recommended plots](#) [New interactive sheet](#)

- Kode ini mempersiapkan dataset lebah untuk analisis lebih lanjut dan pelatihan model dengan cara:
Membaca dan mengonversi tipe data, Menghapus data yang tidak lengkap (missing values), Memastikan hanya gambar yang valid yang akan digunakan.
Langkah ini penting untuk menjaga kualitas data sebelum masuk ke tahap preprocessing dan pelatihan model lebih lanjut.


```

f, ax = plt.subplots(nrows=2, ncols=2, figsize=(12,8))

bees.subspecies.value_counts().plot(kind='bar', ax=ax[0, 0])
ax[0,0].set_ylabel('Count')
ax[0,0].set_title('Subspecies')

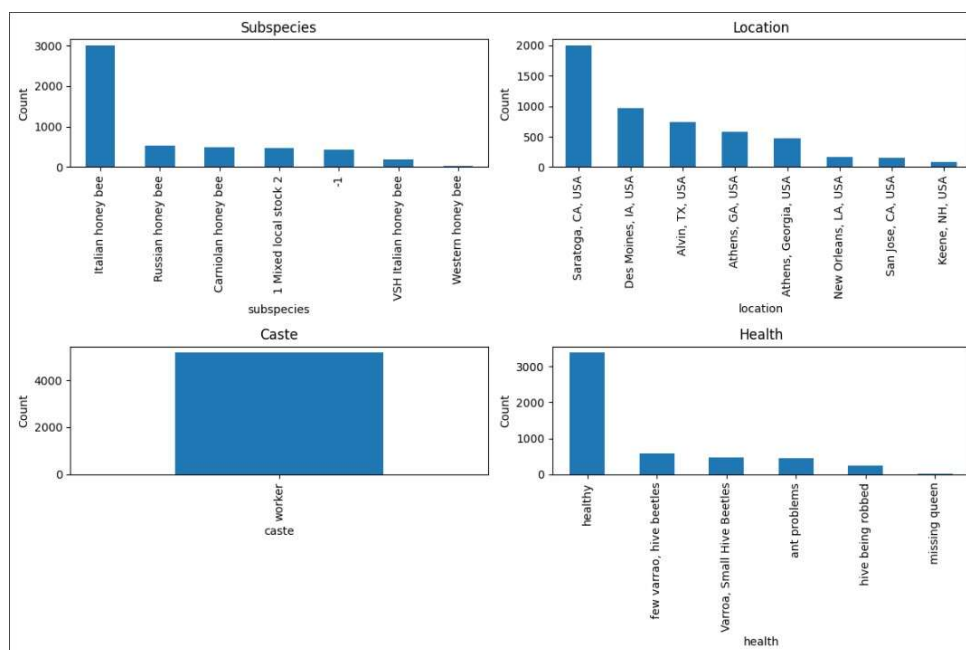
bees.location.value_counts().plot(kind='bar', ax=ax[0, 1])
ax[0,1].set_title('Location')
ax[0,1].set_ylabel('Count')

bees.caste.value_counts().plot(kind='bar', ax=ax[1, 0])
ax[1,0].set_title('Caste')
ax[1,0].set_ylabel('Count')

bees.health.value_counts().plot(kind='bar', ax=ax[1,1])
ax[1,1].set_title('Health')
ax[1,1].set_ylabel('Count')

f.subplots_adjust(hspace=0.7)
f.tight_layout()
plt.show()

```



- Kode di atas digunakan untuk membuat grafik bar (bar plot) yang menampilkan distribusi data dari beberapa kolom penting pada dataset lebah. Kode ini digunakan untuk melakukan eksplorasi data awal (exploratory data analysis) dengan memvisualisasikan distribusi dari beberapa fitur kategorikal penting pada dataset lebah, yaitu: Subspecies (jenis lebah), Location (lokasi asal), Caste (kasta atau peran dalam koloni), Health (kondisi kesehatan lebah).

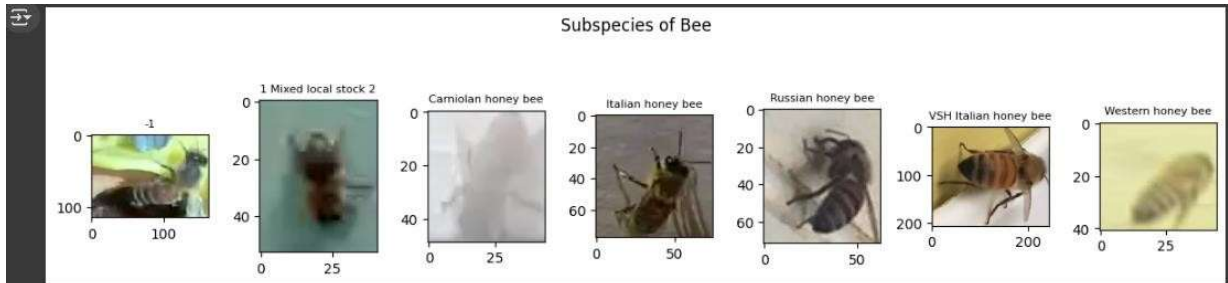
Visualisasi ini membantu untuk memahami pola data, distribusi kategori, dan mengetahui apakah terdapat ketidakseimbangan dalam dataset yang perlu diperhatikan pada langkah selanjutnya dalam proses analisis atau pelatihan model.

```
# Updated import to avoid future deprecation issues

# Select first X subspecies titles
subspecies = bees['subspecies'].cat.categories
f, ax = plt.subplots(nrows=1, ncols=subspecies.size, figsize=(12, 3))
i = 0

# Draw the first found bee of given subspecies
for s in subspecies:
    if s == 'healthy': continue
    file = img_folder + bees[bees['subspecies'] == s].iloc[0]['file']
    im = imageio.imread(file)
    ax[i].imshow(im, resample=True)
    ax[i].set_title(s, fontsize=8)
    i += 1

plt.suptitle("Subspecies of Bee")
plt.tight_layout()
plt.show()
```



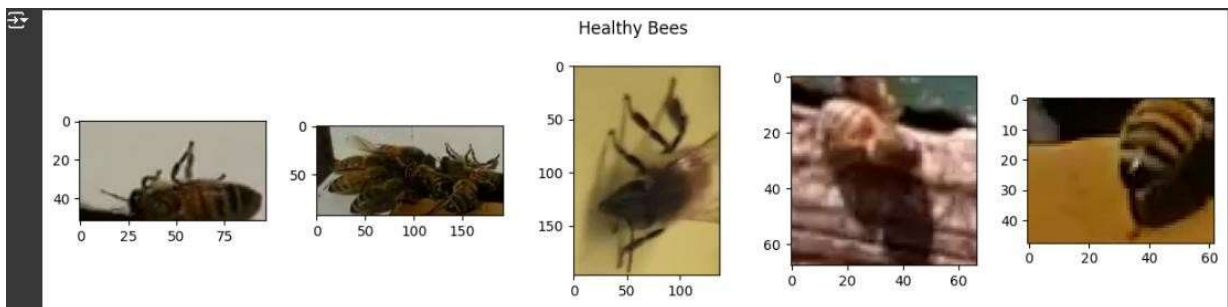
Kode ini bertujuan untuk melakukan eksplorasi data visual dengan menampilkan contoh gambar lebah dari setiap subspecies yang ada dalam dataset. Ini membantu pengguna untuk: Memahami perbedaan visual antar subspecies lebah. Memverifikasi kualitas gambar dalam dataset. Memperoleh gambaran umum dari dataset sebelum melakukan preprocessing dan pelatihan model. Kode ini melewati subspecies 'healthy', sehingga fokus hanya pada subspecies lainnya. Penggunaan `resample=True` dalam `imshow()` bertujuan untuk meningkatkan kualitas gambar saat ditampilkan.

Healthy Bees

```
ncols = 5
healthy = bees[bees['health'] == 'healthy'].sample(ncols)

f, ax = plt.subplots(nrows=1,ncols=ncols, figsize=(12,3))
# Read image of original size from disk, because bees['img'] contains resized numpy array
for i in range(0,5):
    file = img_folder + healthy.iloc[i]['file']
    ax[i].imshow(imageio.imread(file))

plt.suptitle("Healthy Bees")
plt.tight_layout()
plt.show()
```

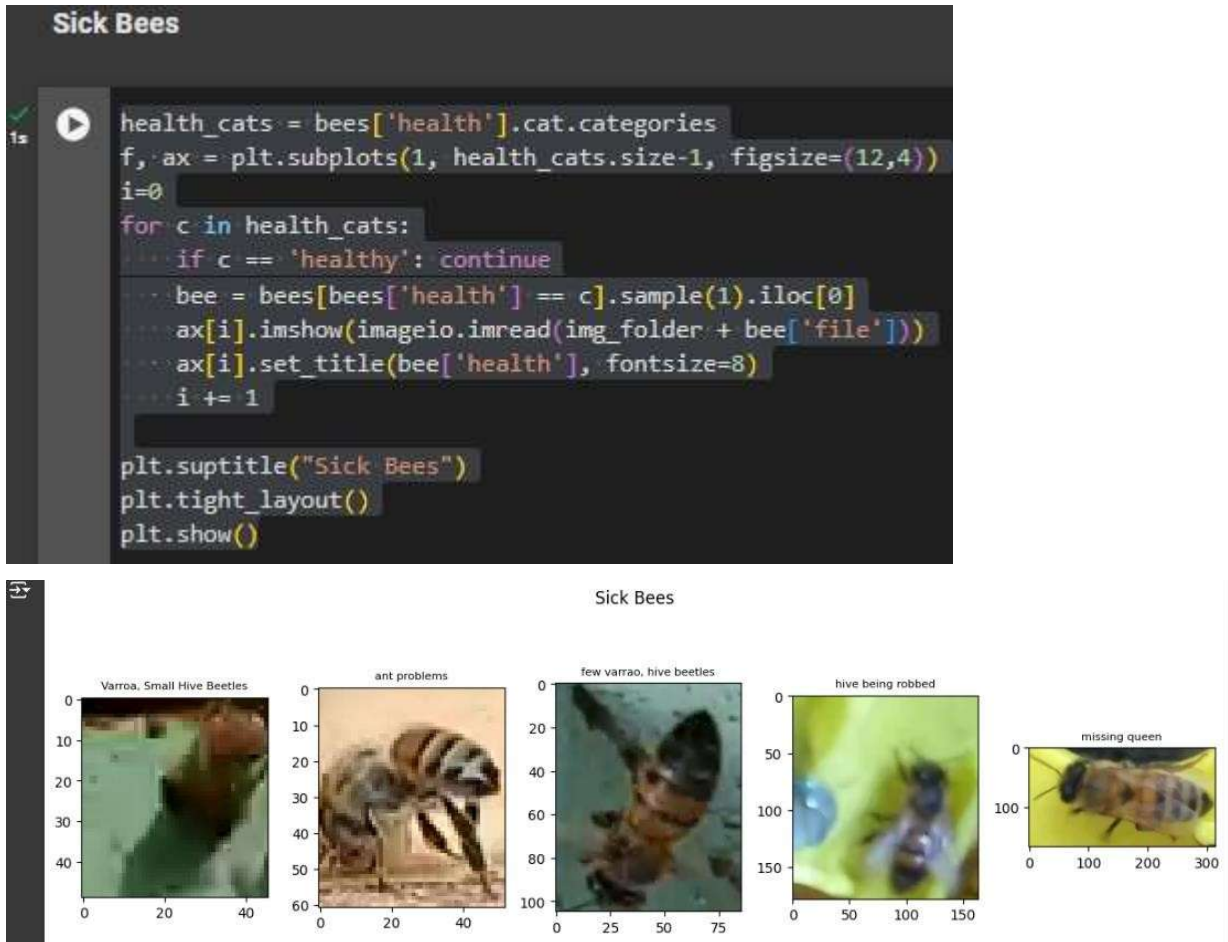


- Kode ini digunakan untuk melakukan eksplorasi visual dengan menampilkan gambar contoh dari lebah sehat dalam dataset. Ini membantu untuk:

Memverifikasi kualitas dan tampilan gambar lebah yang dikategorikan sehat.

Memahami kondisi visual lebah sehat sebagai referensi sebelum melakukan analisis lebih lanjut atau saat membandingkan dengan kondisi kesehatan lain dalam dataset.

Pengambilan sampel menggunakan `.sample(ncols)` memastikan bahwa gambar yang ditampilkan dipilih secara acak, memberikan variasi dalam contoh yang ditampilkan. Dengan menampilkan gambar asli dari disk, kualitas gambar yang ditampilkan tetap baik tanpa melalui proses resize seperti yang mungkin terjadi pada gambar yang sudah diproses.



- Kode ini bertujuan untuk memberikan eksplorasi visual dari berbagai kondisi kesehatan lebah dalam dataset yang tidak sehat (sakit). Hal ini membantu untuk:
Mengenali perbedaan visual antar kondisi kesehatan lebah yang berbeda, seperti infeksi, parasit, atau penyakit lainnya. Memverifikasi data gambar untuk memastikan setiap kondisi kesehatan memiliki representasi yang tepat dalam dataset sebelum melanjutkan ke tahap analisis atau model pelatihan. Menggunakan sampel acak (.sample(1)) memberikan variasi dalam contoh yang ditampilkan setiap kali kode dijalankan.
Fokus visualisasi pada kategori lebah yang tidak sehat memungkinkan pengguna untuk lebih memahami berbagai kondisi penyakit atau masalah yang dihadapi lebah dalam dataset.

```
def split_balance(bees, field_name):
    # Split to train and test before balancing
    train_bees, test_bees = train_test_split(bees, random_state=24)

    # Split train into train and validation datasets
    train_bees, val_bees = train_test_split(train_bees, test_size=0.1, random_state=24)

    # Balance by subspecies for the train dataset
    ncat_bal = int(len(train_bees) / train_bees[field_name].cat.categories.size)

    # Use 'include_groups=False' to avoid the deprecated warning
    train_bees_bal = train_bees.groupby(field_name, as_index=False, observed=True).apply(
        lambda g: g.sample(ncat_bal, replace=True, random_state=24) # Adding random_state for reproducibility
    ).reset_index(drop=True)

    return train_bees_bal, val_bees, test_bees

def plot_balanced(train_bees, train_bees_bal, field_name):
    # Plot before and after balancing
    f, axs = plt.subplots(1, 2, figsize=(8, 4))

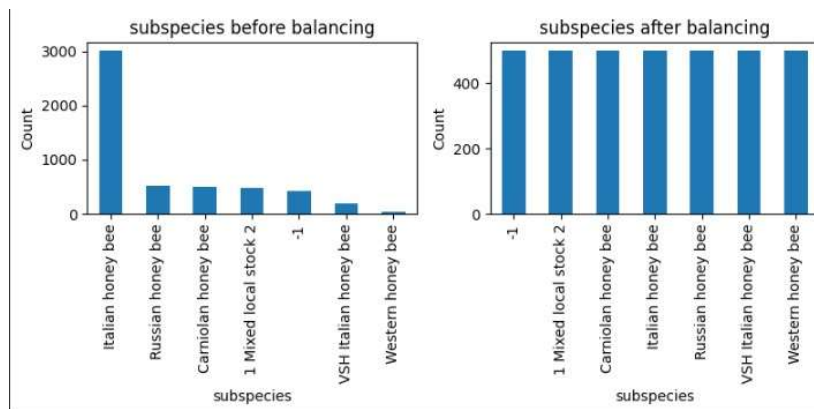
    # Before balancing
    ax = train_bees[field_name].value_counts().plot(kind='bar', ax=axs[0])
    ax.set_title(f'{field_name} before balancing')
    ax.set_ylabel('Count')

    # After balancing
    ax = train_bees_bal[field_name].value_counts().plot(kind='bar', ax=axs[1])
    ax.set_title(f'{field_name} after balancing')
    ax.set_ylabel('Count')

    plt.tight_layout()
    plt.show()

# Split/balance and plot the result
train_bees_bal, val_bees, test_bees = split_balance(bees, 'subspecies')
plot_balanced(bees, train_bees_bal, 'subspecies')

# Use the balanced dataset as main
train_bees = train_bees_bal
```



- Pembagian Dataset: Memisahkan dataset menjadi train, validation, dan test untuk mempersiapkan proses pelatihan dan evaluasi model.

Balancing Dataset: Menyamakan jumlah sampel per kategori subspecies dalam dataset training untuk menghindari bias pada model akibat ketidakseimbangan data.

Visualisasi: Membantu memverifikasi bahwa balancing sudah dilakukan dengan benar, sehingga setiap kategori memiliki jumlah sampel yang sama dalam dataset training.

Manfaat Balancing:

Mengurangi risiko model terlatih secara bias terhadap kategori yang lebih dominan.

Meningkatkan performa dan generalisasi model, terutama ketika dataset awal memiliki ketidakseimbangan yang signifikan antar kategori.


```

# The same way of loading images and one hot encoding will be used in 2 places: subspecies and health CNN.
# Let's put this logic in function here to reuse.
def prepare2train(train_bees, val_bees, test_bees, field_name):
    """
    Load images for features, drop other columns
    One hot encode for label, drop other columns
    @return: image generator, train images, validation images, test images, train labels, validation labels, test labels
    """
    # Bees already splitted to train, validation and test
    # Load and transform images to have equal width/height/channels.
    # read_img function is defined in the beginning to use in both health and subspecies.
    # Use np.stack to get NumPy array for CNN input

    # Train data
    train_X = np.stack(train_bees['file'].apply(read_img))
    train_y = to_categorical(train_bees[field_name].values)
    train_y = pd.get_dummies(train_bees[field_name], drop_first=False)

    # Validation during training data to calc val loss metric
    val_X = np.stack(val_bees['file'].apply(read_img))
    val_y = to_categorical(val_bees[field_name].values)
    val_y = pd.get_dummies(val_bees[field_name], drop_first=False)

    # Test data
    test_X = np.stack(test_bees['file'].apply(read_img))
    test_y = to_categorical(test_bees[field_name].values)
    test_y = pd.get_dummies(test_bees[field_name], drop_first=False)

    # Data augmentation - a little bit rotate, zoom and shift input images.
    generator = ImageDataGenerator(
        featurewise_center=False, # set input mean to 0 over the dataset
        samplewise_center=False, # set each sample mean to 0
        featurewise_std_normalization=False, # divide inputs by std of the dataset
        samplewise_std_normalization=False, # divide each input by its std
        zca_whitening=False, # apply ZCA whitening
        rotation_range=180, # randomly rotate images in the range (degrees, 0 to 180)
        zoom_range = 0.1, # Randomly zoom image
        width_shift_range=0.2, # randomly shift images horizontally (fraction of total width)
        height_shift_range=0.2, # randomly shift images vertically (fraction of total height)
        horizontal_flip=True, # randomly flip images
        vertical_flip=True)
    generator.fit(train_X)
    return (generator, train_X, val_X, test_X, train_y, val_y, test_y)

#train_bees['subspecies'].cat.values[:100]
# Call image preparation and one hot encoding
generator, train_X, val_X, test_X, train_y, val_y, test_y = prepare2train(train_bees, val_bees, test_bees, 'subspecies')

```

- Persiapan Data: Memuat dan memproses gambar lebah serta mengubah label menjadi format yang siap digunakan pada model CNN. One-Hot Encoding: Mengubah label kategori menjadi bentuk numerik yang bisa digunakan untuk klasifikasi multikelas dalam CNN. Augmentasi Data: Meningkatkan variasi data gambar untuk membantu model mengenali berbagai variasi dari gambar lebah yang mungkin terjadi di dunia nyata.
- Manfaat Augmentasi Data:
Mengurangi overfitting dengan memperkenalkan variasi tambahan pada data training. Membantu model dalam generalization dengan belajar dari berbagai modifikasi gambar yang mungkin terjadi pada data nyata (misalnya rotasi, flipping, dan shifting). Augmentasi gambar hanya diterapkan pada train_X (dataset training), sedangkan data validasi (val_X) dan pengujian (test_X) digunakan dalam bentuk aslinya tanpa augmentasi untuk mengevaluasi performa model secara akurat.

```

[12] checkpointer1 = ModelCheckpoint('best_model1.weights.h5'
                                     ,monitor='val_accuracy'
                                     ,verbose=1
                                     ,save_best_only=True
                                     ,save_weights_only=True)

```

- Mencegah Overfitting: Dengan memantau akurasi validasi (val_accuracy), callback ini akan menyimpan bobot model terbaik selama pelatihan, yang mencegah model overfitting pada data training saja. Efisiensi Penyimpanan: Hanya menyimpan bobot ketika performa model meningkat, sehingga menghemat ruang penyimpanan dengan tidak menyimpan bobot yang tidak optimal. Memastikan Reprodutifitas: Setelah pelatihan selesai, bobot terbaik dapat dimuat kembali untuk evaluasi atau prediksi tanpa perlu melakukan pelatihan ulang. ModelCheckpoint adalah cara yang efektif untuk menyimpan model terbaik selama proses pelatihan, yang dapat menghemat waktu dan mencegah overfitting. Menggunakan parameter seperti monitor, save_best_only, dan save_weights_only memungkinkan kontrol yang lebih baik atas penyimpanan model berdasarkan kriteria performa yang diinginkan.

```

# We'll stop training if no improvement after some epochs
earlystopper1 = EarlyStopping(monitor='loss', patience=10, verbose=1)

# Build CNN model
model1=Sequential()
model1.add(Conv2D(6, kernel_size=3, input_shape=(img_width, img_height,3), activation='relu', padding='same'))
model1.add(MaxPool2D(2))
model1.add(Conv2D(12, kernel_size=3, activation='relu', padding='same'))
model1.add(Flatten())
model1.add(Dense(train_y.columns.size, activation='softmax'))
model1.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

# Train
training1 = model1.fit(generator.flow(train_X,train_y, batch_size=60)
                      ,epochs=20
                      ,validation_data=(val_X, val_y)
                      ,steps_per_epoch=50
                      )

```

```

/usr/local/lib/python3.10/dist-packages/keras/src/layers/convolutional/base_conv.py:107: UserWarning: Do not pass an `input_shape` argument to `Conv2D` layers.
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/20
/usr/local/lib/python3.10/dist-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:121: UserWarning: Your `PyDatasetAdapter` class does not implement the `get_data_adapter_config` method.
self._warn_if_super_not_called()
50/50 — 30s 517ms/step - accuracy: 0.2199 - loss: 3.5858 - val_accuracy: 0.6572 - val_loss: 1.1224
Epoch 2/20
50/50 — 10s 260ms/step - accuracy: 0.5698 - loss: 1.0965 - val_accuracy: 0.6572 - val_loss: 1.1224
50/50 — 4s 69ms/step - accuracy: 0.5855 - loss: 1.0558 - val_accuracy: 0.5490 - val_loss: 1.1569
Epoch 3/20
50/50 — 38s 538ms/step - accuracy: 0.6767 - loss: 0.8608 - val_accuracy: 0.6057 - val_loss: 0.9684
Epoch 4/20
50/50 — 3s 57ms/step - accuracy: 0.7582 - loss: 0.6614 - val_accuracy: 0.6057 - val_loss: 1.0479
Epoch 5/20
50/50 — 27s 503ms/step - accuracy: 0.7821 - loss: 0.6534 - val_accuracy: 0.7552 - val_loss: 0.6692
Epoch 6/20
50/50 — 4s 80ms/step - accuracy: 0.8011 - loss: 0.5729 - val_accuracy: 0.7088 - val_loss: 0.7515
Epoch 7/20
50/50 — 36s 492ms/step - accuracy: 0.8244 - loss: 0.5551 - val_accuracy: 0.6959 - val_loss: 0.7890
Epoch 8/20
50/50 — 3s 57ms/step - accuracy: 0.8454 - loss: 0.4776 - val_accuracy: 0.6881 - val_loss: 1.0253
Epoch 9/20
50/50 — 38s 499ms/step - accuracy: 0.8266 - loss: 0.5109 - val_accuracy: 0.6675 - val_loss: 0.8750
Epoch 10/20
50/50 — 3s 57ms/step - accuracy: 0.8424 - loss: 0.4932 - val_accuracy: 0.7088 - val_loss: 0.7669
Epoch 11/20
50/50 — 26s 499ms/step - accuracy: 0.8510 - loss: 0.4767 - val_accuracy: 0.7938 - val_loss: 0.5632
Epoch 12/20
50/50 — 3s 57ms/step - accuracy: 0.8842 - loss: 0.4055 - val_accuracy: 0.6598 - val_loss: 0.8661
Epoch 13/20
50/50 — 27s 519ms/step - accuracy: 0.8559 - loss: 0.4141 - val_accuracy: 0.7165 - val_loss: 0.7279
Epoch 14/20
50/50 — 3s 57ms/step - accuracy: 0.8900 - loss: 0.3592 - val_accuracy: 0.7706 - val_loss: 0.5940
Epoch 15/20
50/50 — 26s 504ms/step - accuracy: 0.8711 - loss: 0.3965 - val_accuracy: 0.7448 - val_loss: 0.6780
Epoch 16/20
50/50 — 3s 57ms/step - accuracy: 0.8626 - loss: 0.4493 - val_accuracy: 0.6881 - val_loss: 0.9413
Epoch 17/20
50/50 — 26s 501ms/step - accuracy: 0.8688 - loss: 0.3849 - val_accuracy: 0.7345 - val_loss: 0.7478
Epoch 18/20
50/50 — 3s 52ms/step - accuracy: 0.8900 - loss: 0.3350 - val_accuracy: 0.7964 - val_loss: 0.5226
Epoch 19/20
50/50 — 26s 495ms/step - accuracy: 0.8846 - loss: 0.3478 - val_accuracy: 0.8531 - val_loss: 0.4503
Epoch 20/20
50/50 — 3s 58ms/step - accuracy: 0.8811 - loss: 0.3221 - val_accuracy: 0.7345 - val_loss: 0.7572

```

- Pelatihan Model CNN: Model CNN ini digunakan untuk klasifikasi gambar lebah berdasarkan berbagai kategori seperti subspecies atau kesehatan.

Menggunakan EarlyStopping: Dengan EarlyStopping yang memiliki patience=10, pelatihan akan dihentikan jika loss tidak menunjukkan perbaikan setelah 10 epoch berturut-turut. Ini berguna untuk menghindari overfitting dan menghemat waktu pelatihan.

Data Augmentasi: Penggunaan generator dengan augmentasi data memungkinkan model untuk dilatih pada gambar yang lebih beragam, sehingga memperbaiki generalisasi model.

Kesimpulan:

Kode ini melatih model CNN untuk klasifikasi gambar dengan memantau loss dan menggunakan EarlyStopping untuk menghentikan pelatihan jika tidak ada peningkatan setelah beberapa epoch.

Pelatihan dilakukan dengan batch size 60 dan menggunakan data augmentasi untuk meningkatkan keragaman data pelatihan.


```

# This is a function to use in Bee subspecies and health evaluation
def eval_model(training, model, test_X, test_y, field_name):
    """
    Model evaluation: plots, classification report
    @param training: model training history
    @param model: trained model
    @param test_X: features
    @param test_y: labels
    @param field_name: label name to display on plots
    """
    ## Trained model analysis and evaluation
    f, ax = plt.subplots(2,1, figsize=(5,5))
    ax[0].plot(training.history['loss'], label="Loss")
    ax[0].plot(training.history['val_loss'], label="Validation loss")
    ax[0].set_title('%s: loss' % field_name)
    ax[0].set_xlabel('Epoch')
    ax[0].set_ylabel('Loss')
    ax[0].legend()

    # Accuracy
    ax[1].plot(training.history['accuracy'], label="Accuracy")
    ax[1].plot(training.history['val_accuracy'], label="Validation accuracy")
    ax[1].set_title('%s: accuracy' % field_name)
    ax[1].set_xlabel('Epoch')
    ax[1].set_ylabel('Accuracy')
    ax[1].legend()
    plt.tight_layout()
    plt.show()

    # Accuracy by subspecies
    test_pred = model.predict(test_X)

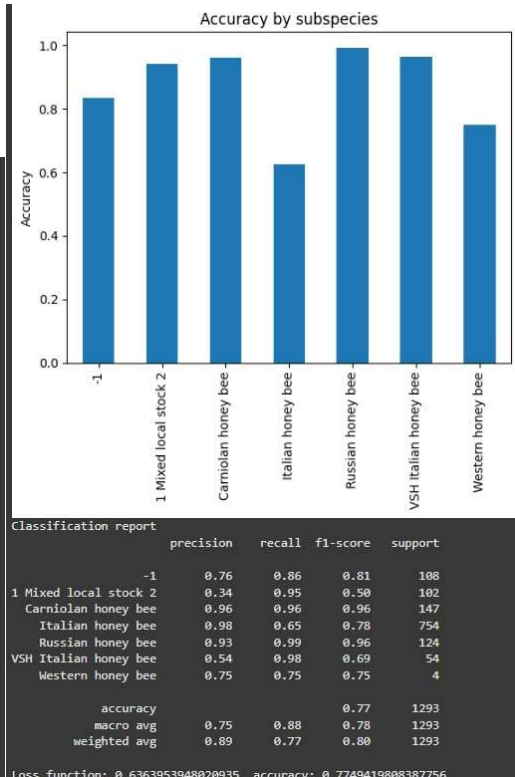
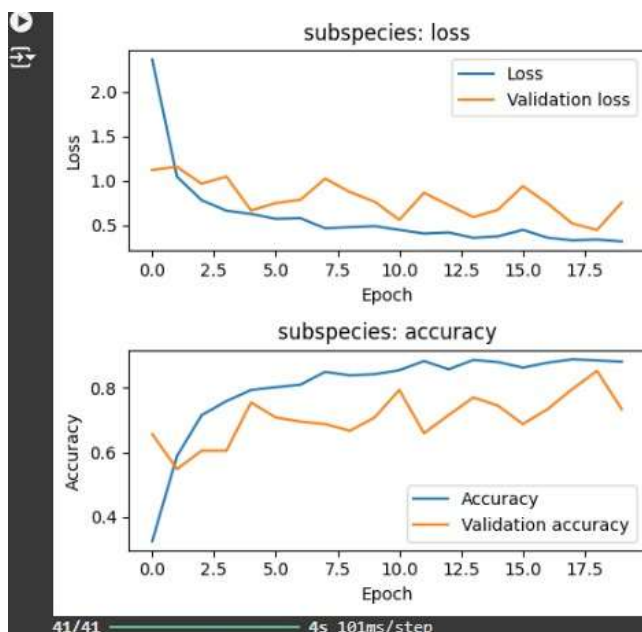
    acc_by_subspecies = np.logical_and((test_pred > 0.5), test_y).sum()/test_y.sum()
    acc_by_subspecies.plot(kind='bar', title='Accuracy by %s' % field_name)
    plt.ylabel('Accuracy')
    plt.show()

    # Print metrics
    print("Classification report")
    test_pred = np.argmax(test_pred, axis=1)
    test_truth = np.argmax(test_y.values, axis=1)
    print(metrics.classification_report(test_truth, test_pred, target_names=test_y.columns))

    # Loss function and accuracy
    test_res = model.evaluate(test_X, test_y.values, verbose=0)
    print('Loss function: %s, accuracy: %s' % test_res[0], test_res[1])

    # Call evaluation function
    eval_model(training1, model1, test_X, test_y, 'subspecies')

```



- **Fungsi Evaluasi:** Fungsi `eval_model` ini memberikan analisis visual dan metrik kualitatif untuk menilai performa model, termasuk loss, accuracy, dan classification report. **Loss dan Accuracy:** Grafik yang menunjukkan perbandingan antara nilai loss dan akurasi di data pelatihan dan validasi memberikan gambaran apakah model mengalami overfitting atau tidak. **Akurasi Berdasarkan Kategori:** Menampilkan akurasi per kategori (subspecies atau health) membantu memverifikasi apakah model bekerja dengan baik pada setiap kategori. **Classification Report dan Evaluasi:** Laporan klasifikasi menyediakan metrik yang lebih mendalam dan evaluasi keseluruhan dengan precision, recall, dan f1-score. Dengan demikian, evaluasi ini memberikan wawasan lengkap tentang kualitas model dalam mengklasifikasikan gambar lebah berdasarkan subspecies atau kondisi kesehatan.



- **Penyeimbangan Dataset Berdasarkan Kesehatan:**
Kode di atas memulai dengan membagi dataset bees menjadi tiga bagian: train, validation, dan test dengan menggunakan fungsi `split_balance(bees, 'health')`. Fungsi ini tidak hanya membagi data, tetapi juga menyeimbangkan data berdasarkan kategori health (sehat atau sakit). Proses penyeimbangan dilakukan dengan melakukan oversampling pada kategori-kategori yang kurang dalam dataset pelatihan agar jumlah data per kategori lebih merata, menggunakan `groupby` dan `apply` pada field health.

Hasil dari pembagian ini adalah tiga dataset: `train_bees_bal`, `val_bees`, dan `test_bees`, di mana `train_bees_bal` adalah dataset pelatihan yang sudah diseimbangkan.

Visualisasi Sebelum dan Setelah Penyeimbangan:

Fungsi `plot_balanced(train_bees, train_bees_bal, 'health')` digunakan untuk menampilkan visualisasi dari distribusi data di seluruh kategori `health` (sehat dan sakit) sebelum dan sesudah penyeimbangan.

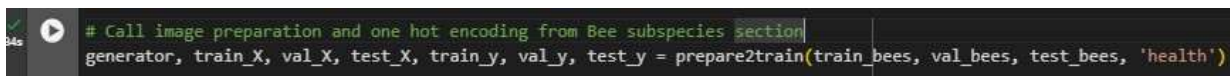
Grafik yang dihasilkan akan menunjukkan bagaimana distribusi label kesehatan berubah setelah proses penyeimbangan, memberikan gambaran apakah data sudah seimbang antara kategori.

Menggunakan Dataset yang Telah Diseimbangkan untuk Pelatihan:

Setelah menyeimbangkan dataset, variabel `train_bees_bal` diatur kembali menjadi dataset utama untuk pelatihan model (`train_bees`), yang sekarang berisi data yang telah diseimbangkan berdasarkan kategori `health`.

- **Penyusunan Dataset:** Fungsi `split_balance` membantu membagi dan menyeimbangkan dataset berdasarkan kategori `health` sebelum digunakan dalam pelatihan.

Visualisasi Distribusi: Dengan `plot_balanced`, distribusi data di seluruh kategori `health` sebelum dan sesudah penyeimbangan divisualisasikan untuk memastikan bahwa model akan dilatih dengan data yang seimbang. **Penggunaan Dataset yang Diseimbangkan:** Dataset pelatihan yang sudah diseimbangkan (`train_bees_bal`) digunakan untuk melatih model, sehingga membantu menghindari masalah ketidakseimbangan kelas yang dapat menyebabkan model cenderung memilih kelas mayoritas.



```
# Call image preparation and one hot encoding from Bee subspecies section
generator, train_X, val_X, test_X, train_y, val_y, test_y = prepare2train(train_bees, val_bees, test_bees, 'health')
```

- **Persiapan Gambar dan Label:** Gambar-gambar telah diproses (diubah ukurannya), dan label telah diubah menjadi format one-hot encoding untuk keperluan pelatihan model CNN. **Augmentasi Data:** Augmentasi gambar diterapkan untuk memperkaya dataset dan meningkatkan kemampuan generalisasi model. **Output:** Fungsi ini mengembalikan dataset yang siap digunakan dalam pelatihan model CNN, yang mencakup fitur (gambar) dan label (`health` yang sudah dikodekan).

```
# Save the best model during the training
checkpointer2 = ModelCheckpoint('best_model2.weights.h5',
                                monitor='val_acc',
                                verbose=1,
                                save_best_only=True,
                                save_weights_only=True)
```

- Fungsi ModelCheckpoint: Digunakan untuk menyimpan bobot model terbaik selama pelatihan, berdasarkan peningkatan akurasi validasi. Efisiensi Penyimpanan: Menggunakan save_weights_only=True memastikan bahwa hanya bobot model yang disimpan, bukan keseluruhan model, yang menghemat ruang penyimpanan. Penggunaan val_acc: Menyimpan model hanya jika ada peningkatan akurasi pada data validasi, sehingga memastikan hanya model dengan kinerja terbaik yang disimpan.

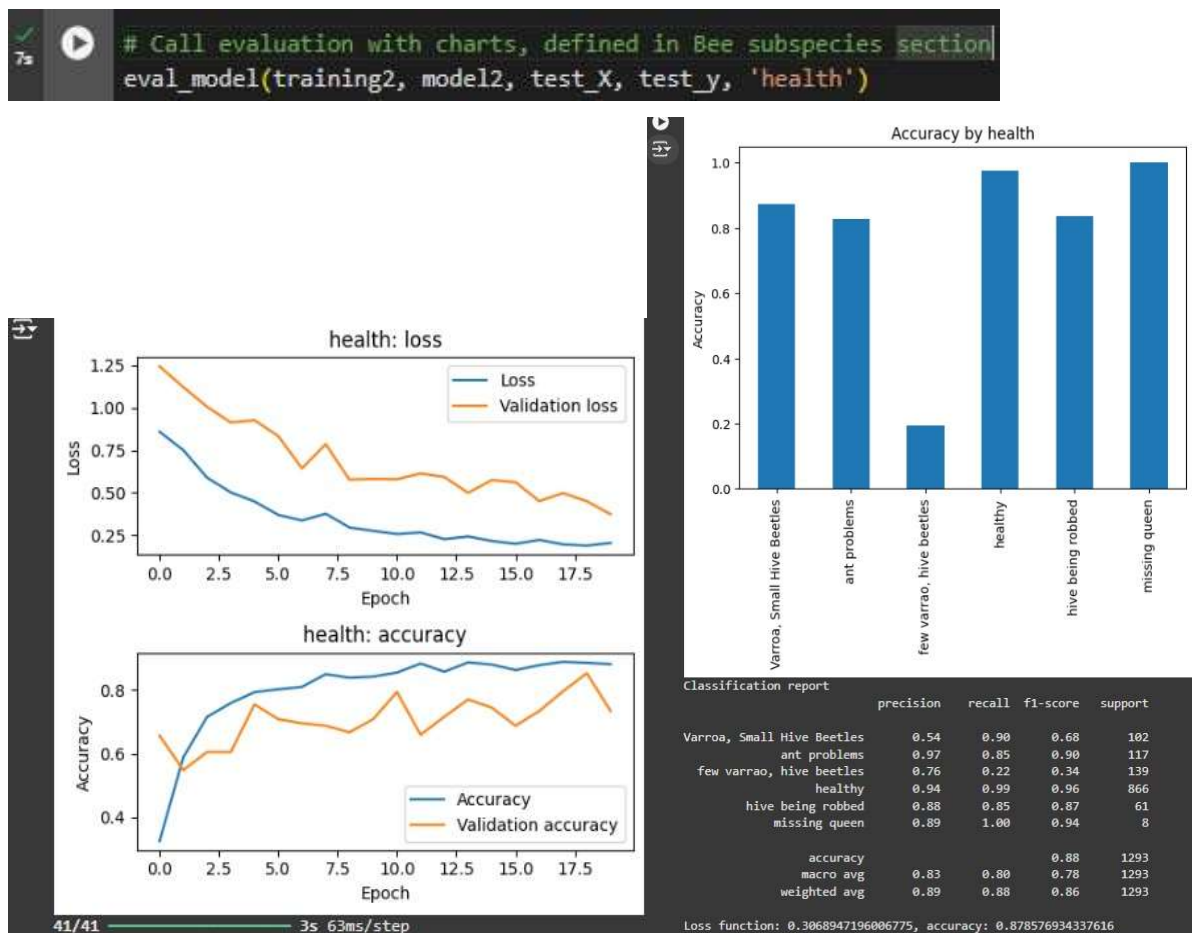
```
# We'll stop training if no improvement after some epochs
earlystopper2 = EarlyStopping(monitor='val_acc', patience=10, verbose=1)

# Build CNN model
model2=Sequential()
model2.add(Conv2D(5, kernel_size=3, input_shape=(img_width, img_height,3), activation='relu', padding='same'))
model2.add(MaxPool2D(2))
model2.add(Conv2D(10, kernel_size=3, activation='relu', padding='same'))
model2.add(Flatten(1))
model2.add(Dense(train_y.columns.size, activation='softmax'))
model2.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

# Train
training2 = model2.fit(generator.flow(train_X,train_y, batch_size=50)
                        ,epochs=20
                        ,validation_data=(val_X, val_y)
                        ,steps_per_epoch=50)
```

```
/usr/local/lib/python3.10/dist-packages/keras/src/layers/convolutional/base_conv.py:107: UserWarning: Do not pass an
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/20
/usr/local/lib/python3.10/dist-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:121: UserWarning: Your
self.warn if super not called()
50/50 — 22s 402ms/step - accuracy: 0.6701 - loss: 1.0010 - val_accuracy: 0.6727 - val_loss: 1.2440
Epoch 2/20
50/50 — 14s 470ms/step - accuracy: 0.7785 - loss: 0.7811 - val_accuracy: 0.6624 - val_loss: 1.1215
Epoch 3/20
50/50 — 11s 211ms/step - accuracy: 0.7800 - loss: 0.7625 - val_accuracy: 0.6624 - val_loss: 1.0059
Epoch 4/20
50/50 — 29s 300ms/step - accuracy: 0.8147 - loss: 0.6221 - val_accuracy: 0.6624 - val_loss: 1.0059
Epoch 5/20
50/50 — 8s 159ms/step - accuracy: 0.8482 - loss: 0.4992 - val_accuracy: 0.6933 - val_loss: 0.9133
Epoch 6/20
50/50 — 20s 380ms/step - accuracy: 0.8472 - loss: 0.4862 - val_accuracy: 0.6959 - val_loss: 0.9267
Epoch 7/20
50/50 — 7s 138ms/step - accuracy: 0.8613 - loss: 0.3751 - val_accuracy: 0.7062 - val_loss: 0.8332
Epoch 8/20
50/50 — 34s 384ms/step - accuracy: 0.8846 - loss: 0.3314 - val_accuracy: 0.7629 - val_loss: 0.6435
Epoch 9/20
50/50 — 7s 140ms/step - accuracy: 0.8651 - loss: 0.3793 - val_accuracy: 0.7191 - val_loss: 0.7854
Epoch 10/20
50/50 — 34s 382ms/step - accuracy: 0.8986 - loss: 0.3126 - val_accuracy: 0.7861 - val_loss: 0.5771
Epoch 11/20
50/50 — 7s 134ms/step - accuracy: 0.9101 - loss: 0.2722 - val_accuracy: 0.7835 - val_loss: 0.5806
Epoch 12/20
50/50 — 22s 411ms/step - accuracy: 0.9194 - loss: 0.2670 - val_accuracy: 0.7732 - val_loss: 0.5790
Epoch 13/20
50/50 — 7s 127ms/step - accuracy: 0.9012 - loss: 0.2863 - val_accuracy: 0.7784 - val_loss: 0.6131
Epoch 14/20
50/50 — 34s 395ms/step - accuracy: 0.9230 - loss: 0.2256 - val_accuracy: 0.7732 - val_loss: 0.5922
Epoch 15/20
50/50 — 6s 117ms/step - accuracy: 0.9253 - loss: 0.2302 - val_accuracy: 0.7912 - val_loss: 0.4990
Epoch 16/20
50/50 — 22s 424ms/step - accuracy: 0.9272 - loss: 0.2226 - val_accuracy: 0.7680 - val_loss: 0.5745
Epoch 17/20
50/50 — 6s 119ms/step - accuracy: 0.9397 - loss: 0.2041 - val_accuracy: 0.7861 - val_loss: 0.5612
Epoch 18/20
50/50 — 36s 456ms/step - accuracy: 0.9373 - loss: 0.2037 - val_accuracy: 0.8222 - val_loss: 0.4504
Epoch 19/20
50/50 — 6s 118ms/step - accuracy: 0.9450 - loss: 0.1893 - val_accuracy: 0.8119 - val_loss: 0.4975
Epoch 20/20
50/50 — 21s 396ms/step - accuracy: 0.9490 - loss: 0.1829 - val_accuracy: 0.8325 - val_loss: 0.4494
Epoch 28/20
50/50 — 10s 192ms/step - accuracy: 0.9347 - loss: 0.2020 - val_accuracy: 0.8351 - val_loss: 0.3733
```

- EarlyStopping digunakan untuk menghentikan pelatihan lebih awal jika tidak ada perbaikan pada akurasi validasi, mencegah overfitting. Model CNN dibangun untuk mengklasifikasikan gambar lebah berdasarkan kategori health (sehat atau sakit). Augmentasi data diterapkan untuk meningkatkan keberagaman data pelatihan dan mengurangi risiko overfitting. Kompilasi dan pelatihan dilakukan dengan menggunakan algoritma optimasi Adam dan fungsi kerugian yang sesuai untuk klasifikasi multi-kelas.



- Grafik yang menunjukkan perkembangan loss dan akurasi selama pelatihan. Grafik yang menunjukkan akurasi model untuk setiap kategori health (misalnya, sehat atau sakit). Laporan klasifikasi yang mencakup precision, recall, dan F1-score untuk setiap kelas. Evaluasi loss dan accuracy pada data uji untuk mengukur kinerja akhir model. Jika semuanya berjalan dengan baik, fungsi ini memberikan gambaran yang lebih jelas mengenai kinerja model dalam memprediksi kategori kesehatan lebah.


```

# Common function for visualization of kernels
def visualize_layer_kernels(img, conv_layer, title):
    """
    Displays how input sample image looks after convolution by each kernel
    :param img: Sample image array
    :param conv_layer: Layer of Conv2D type
    :param title: Text to display on the top
    """
    # Extract kernels from given layer
    weights1 = conv_layer.get_weights()
    kernels = weights1[0]
    kernels_num = kernels.shape[3]

    # Each row contains 3 images: kernel, input image, output image
    f, ax = plt.subplots(kernels_num, 3, figsize=(7, kernels_num*2))

    for i in range(0, kernels_num):
        # Get kernel from the layer and draw it
        kernel=kernels[:, :, 3, i]
        ax[i][0].imshow((kernel * 255).astype(np.uint8), vmin=0, vmax=255)
        ax[i][0].set_title("Kernel %d" % i, fontsize = 9)

        # Get and draw sample image from test data
        ax[i][1].imshow((img * 255).astype(np.uint8), vmin=0, vmax=255)
        ax[i][1].set_title("Before", fontsize=8)

        # Filtered image - apply convolution
        img_filt = scipy.ndimage.filters.convolve(img, kernel)
        ax[i][2].imshow((img_filt * 255).astype(np.uint8), vmin=0, vmax=255)
        ax[i][2].set_title("After", fontsize=8)

    plt.suptitle(title)
    plt.tight_layout()
    plt.subplots_adjust(top=0.93)
    plt.show()

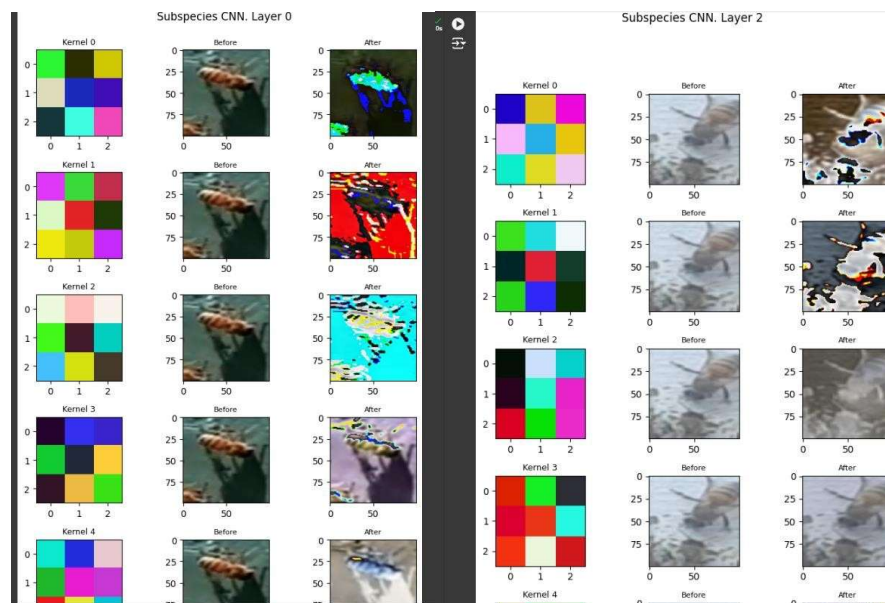
```

- Fungsi `visualize_layer_kernels` membantu memahami bagaimana lapisan konvolusional di Convolutional Neural Network (CNN) memproses gambar masukan. Ini memvisualisasikan efek setiap kernel di lapisan konvolusional pada gambar masukan, dan menampilkan yang berikut: Visualisasi Kernel: Filter aktual (kernel) yang digunakan oleh lapisan konvolusional. Gambar Masukan: Gambar sampel sebelum konvolusi apa pun diterapkan. Gambar yang Difilter: Hasil penerapan filter konvolusi ke gambar masukan, menampilkan fitur yang diekstraksi (misalnya tepi, tekstur). Wawasan Utama: Visualisasi Kernel: Setiap kernel belajar mendeteksi fitur tertentu, seperti tepi, sudut, atau tekstur. Visualisasi kernel menunjukkan bagaimana jaringan mencoba menangkap fitur tingkat rendah. Perbandingan Input dan Output: Dengan membandingkan gambar masukan dan versi yang difilter setelah konvolusi, kita dapat melihat bagaimana setiap kernel menyoroti aspek gambar yang berbeda. Beberapa kernel mungkin menyorot bagian tepinya, sementara kernel lainnya mungkin fokus pada pola atau warna yang lebih kompleks. Fungsionalitas Lapisan: Memahami bagaimana masing-masing kernel mempengaruhi gambar membantu dalam menafsirkan bagaimana jaringan memproses data pada lapisan yang berbeda. Lapisan awal biasanya berfokus pada fitur tingkat rendah (misalnya tepian), sedangkan lapisan yang lebih dalam mungkin menangkap fitur yang lebih abstrak.

- Kesimpulan: Fungsi ini menyediakan alat penting untuk menafsirkan model CNN, memungkinkan pengguna memeriksa secara visual dan memahami peran kernel konvolusional dalam ekstraksi fitur. Hal ini sangat berguna untuk debugging model, memahami fitur yang dipelajari, dan meningkatkan transparansi dalam pemrosesan gambar berbasis CNN.

```
# Take sample image to visualize convolution
idx = random.randint(0,len(test_X)-1)
img = test_X[idx,:,:,:]
# Take 1st convolutional layer and look at it's filters
conv1 = model1.layers[0]
img = visualize_layer_kernels(img, conv1, "Subspecies CNN. Layer 0")

# Take sample image to visualize convolutoin
idx = random.randint(0,len(test_y)-1)
img = test_X[idx,:,:,:]
# Take another convolutional layer and look at it's filters
conv2 = model1.layers[2]
res = visualize_layer_kernels(img, conv2, "Subspecies CNN. Layer 2")
```



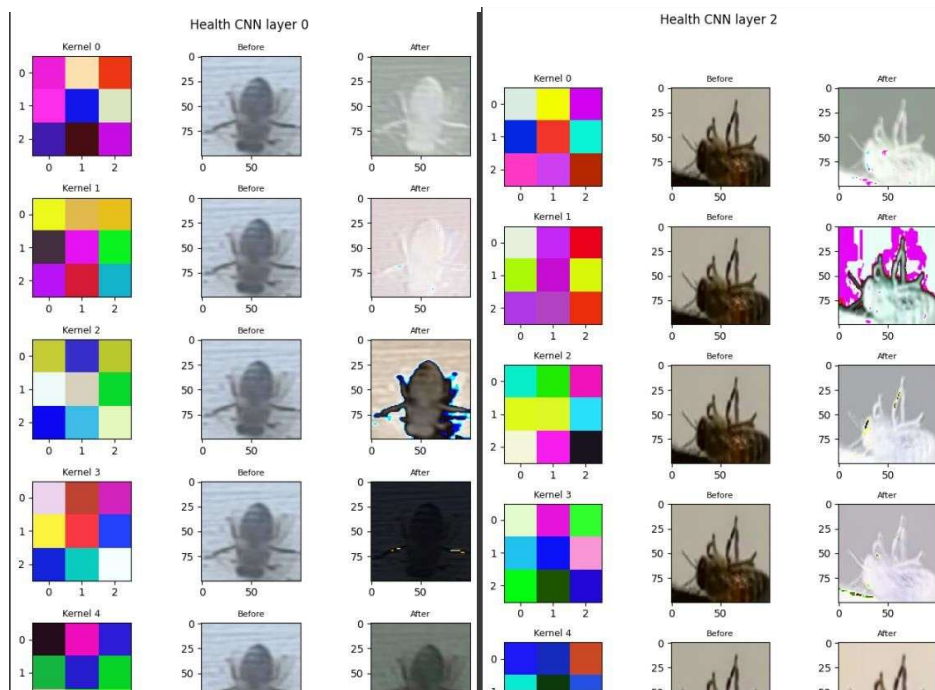
- Lapisan 0 (Konv1): Lapisan ini kemungkinan akan menyorot fitur tingkat rendah yang sederhana seperti tepi, garis, atau tekstur dasar. Lapisan 2 (Konv2): Pada tahap ini, fitur-fitur yang terdeteksi oleh lapisan pertama digabungkan menjadi representasi yang lebih kompleks. Lapisan ini kemungkinan akan fokus pada pola yang lebih abstrak seperti bentuk, wilayah, atau tekstur yang membantu dalam klasifikasi. Visualisasi ini memberikan wawasan berharga tentang bagaimana jaringan mempelajari dan memproses fitur gambar, dan merupakan alat penting untuk memahami dan menafsirkan CNN.

```

# Take sample image to visualize convolution
idx = random.randint(0,len(test_X)-1)
img = test_X[idx,:,:,:]
#img = img[:,:,:]
# Take 1st convolutional layer and look at it's filters
conv1 = model2.layers[0]
visualize_layer_kernels(img, conv1, "Health CNN layer 0")

# Take sample image to visualize convolution
idx = random.randint(0,len(test_X)-1)
img = test_X[idx,:,:,:]
# Take another convolutional layer and look at it's filter
conv2 = model2.layers[2]
visualize_layer_kernels(img, conv2, "Health CNN layer 2")

```



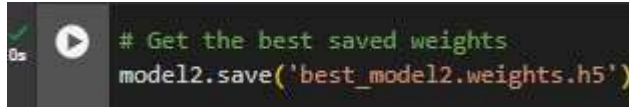
- Lapisan 0 (Konv1): Lapisan konvolusional pertama kemungkinan akan fokus pada pola dasar, seperti tepi, garis, atau tekstur sederhana. Lapisan 2 (Konv2): Lapisan kedua akan memproses pola yang terdeteksi oleh lapisan pertama, dan filternya kemungkinan besar akan fokus pada struktur yang lebih kompleks seperti bentuk atau wilayah, sehingga memungkinkan ekstraksi fitur yang lebih abstrak. Dengan memvisualisasikan hasil lapisan ini, Anda dapat memperoleh wawasan tentang bagaimana model mempelajari representasi berbeda dari gambar masukan, yang berguna untuk memahami perilaku model selama proses ekstraksi fitur.

```

# Get the best saved weights
model1.save('best_model1.weights.h5')

```


- Dengan cara ini, hanya bobot model yang akan disimpan, yang biasanya Anda inginkan saat menyimpan bobot model 1 dengan performa terbaik selama pelatihan menggunakan callback seperti ModelCheckpoint.

A code snippet from a Jupyter Notebook. It features a green checkmark icon and a play button icon on the left. The code is written in a dark-themed editor with syntax highlighting: a comment line in green and a function call in white and red.

```
# Get the best saved weights  
model2.save('best_model2.weights.h5')
```

- Dengan cara ini, hanya bobot model yang akan disimpan, yang biasanya Anda inginkan saat menyimpan bobot model 2 dengan performa terbaik selama pelatihan menggunakan callback seperti ModelCheckpoint.

BAB IV

PENUTUP

4.1 Kesimpulan

Proyek ini bertujuan untuk mengembangkan model Convolutional Neural Network (CNN) untuk mengklasifikasikan gambar lebah berdasarkan subspecies dan kesehatan mereka. Dalam rangkaian ini, berbagai langkah dilakukan, mulai dari pra-pemrosesan data, pelatihan model, hingga evaluasi hasil untuk memastikan akurasi dan efektivitas model. Berikut adalah kesimpulan utama dari proyek ini:

Persiapan Data: Dataset lebah yang digunakan terdiri dari gambar lebah yang dikategorikan menurut subspecies dan kesehatan. Pra-pemrosesan gambar dilakukan dengan melakukan resize gambar agar sesuai dengan ukuran input yang diperlukan oleh CNN. Labeling gambar dilakukan menggunakan one-hot encoding untuk mengkonversi kategori (subspecies atau kesehatan) menjadi format yang dapat diterima oleh model.

Pembagian Dataset: Dataset dibagi menjadi tiga bagian: train, validation, dan test.

Proses penyeimbangan (balancing) dataset dilakukan dengan menggunakan oversampling agar jumlah gambar dari setiap kelas (subspecies atau status kesehatan) seimbang, untuk menghindari bias model.

Arsitektur Model CNN: Dua model CNN terpisah dibangun: satu untuk mengklasifikasikan subspecies dan satu lagi untuk mengklasifikasikan kesehatan lebah.

Model-model ini terdiri dari beberapa lapisan konvolusional (Conv2D) dan lapisan pooling (MaxPool2D), diikuti oleh lapisan fully connected untuk menghasilkan prediksi kelas.

Fungsi aktivasi ReLU digunakan di lapisan konvolusional, dan softmax digunakan di lapisan output untuk klasifikasi multi-kelas.

Pelatihan Model: Selama pelatihan, dilakukan data augmentation untuk meningkatkan keragaman gambar dan mencegah overfitting, dengan memutar, menggeser, dan membalik gambar secara acak.

ModelCheckpoint digunakan untuk menyimpan bobot terbaik selama pelatihan berdasarkan performa pada data validasi. EarlyStopping diterapkan untuk menghentikan pelatihan jika tidak ada perbaikan signifikan dalam akurasi, sehingga menghindari overfitting.

Evaluasi Model: Evaluasi dilakukan menggunakan akurasi dan loss pada data validasi dan test. Model yang telah dilatih diuji untuk memprediksi kelas yang benar, dan hasilnya dibandingkan dengan label yang benar. Classification report yang berisi precision, recall, dan f1-score untuk setiap kelas juga dihitung untuk menilai performa model secara lebih mendalam.

Visualisasi Fitur: Visualisasi kernel pada lapisan konvolusional model digunakan untuk memahami bagaimana fitur dari gambar diproses dan apa yang dipelajari oleh model.

Gambar asli, kernel, dan gambar setelah konvolusi ditampilkan untuk menunjukkan bagaimana lapisan pertama dan kedua berinteraksi dengan input gambar.

Penyimpanan Model: Setelah pelatihan selesai, model disimpan dalam format HDF5 untuk digunakan di masa depan. Dengan cara ini, model yang sudah dilatih dapat dimuat kembali tanpa perlu melakukan pelatihan ulang.

4.2 Saran

Untuk meningkatkan hasil lebih lanjut, model bisa dioptimalkan dengan eksperimen lebih lanjut pada arsitektur CNN, seperti menambahkan lebih banyak lapisan konvolusional, atau menggunakan transfer learning dengan model pre-trained (misalnya, VGG16 atau ResNet) yang dapat mempercepat konvergensi dan meningkatkan performa pada dataset yang lebih kecil.

Secara keseluruhan, proyek ini menunjukkan bahwa CNN adalah pendekatan yang efektif untuk pengenalan gambar dan klasifikasi dalam konteks pertanian dan ilmu biologi, khususnya dalam mempelajari berbagai karakteristik lebah berdasarkan gambar.

DAFTAR PUSTAKA

1. <https://arxiv.org/format/1511.08458> An Introduction to Convolutional Neural Networks Keiron O'Shea, Ryan Nash
2. <https://cs.nju.edu.cn/wujx/paper/CNN.pdf> Introduction to Convolutional Neural Networks Jianxin Wu LAMDA Group National Key Lab for Novel Software Technology Nanjing University, China
3. <https://arxiv.org/abs/1605.09081> Understanding Convolutional Neural Networks Jayanth Koushik
4. <https://link.springer.com/article/10.1007/s13592-022-00918-5>
5. <https://link.springer.com/article/10.1007/s13042-018-0811-z> Towards improving the convolutional neural networks for deep learning using the distributed artificial bee colony method
6. <https://ieeexplore.ieee.org/abstract/document/8875886> Development of CNN architecture for Honey Bees Disease Condition
7. https://link.springer.com/chapter/10.1007/978-94-009-1900-6_1 What is Artificial Intelligence?